IKBT User Manual

Blake Hannaford with help from Claude Opus5.5

November 5, 2026

Who this is for: you have a robot, you have run IKBT on it, and you want to *use* the result: read the report, call the inverse kinematics from your own program, validate the solution, etc. For knowledge of how IKBT works inside, see the JAIR paper

Requirements:

Python 3, `sympy`, `numpy`; a LaTeX distribution to compile the report; `g++` (C++17) to build the C++.

Contents

1 Running IKBT **2**

2 Words and conventions **2**

3 Which method did my robot get? **3**

4 The files, at a glance **3**

5 Python Code Generation **4**

5.1 Symbolic 4

5.2 One-variable 5

5.3 Hybrid — two calls, on purpose 5

5.4 Forward kinematics and Jacobian (every method) 6

6 C++ Code Generation **6**

7 The LaTeX report **7**

8 Validating The Results with Closed Loop Testing **7**

9 Other outputs **8**

10 Regenerating and cleaning up **9**

11 When something looks wrong **9**

1 Running IKBT

The first step is defining your robot by name in `ikbtfunctions/ik_robots.py` (`robot_params()`): its DH table, its joint variables, and numeric values (`pvals`) for its link lengths. To add your robot,

1. Give it a name which does not appear in the name list (`ikbtfunctions/ik_robots.py: ROBOT_LIST`).
2. Add the new name to `ROBOT_LIST`.
3. Copy one of the existing robot definitions (e.g. `if(name == 'KinovaLite'):`), and modify it to your specific parameters.
4. The `vv[]` vector should read `1` for a rotary joint and `0` for a prismatic joint.
5. Your DH table must have 6 rows. If your robot as ≤ 6 DOF, add rows of all zeros.

Give every parameter a numeric value in `pvals`. Everything runs **from the IKBT root directory**:

```
python3 ikSolver.py Puma      # solve one robot (in this case ``Puma'')
python3 fkOnly.py Puma        # forward kinematics and Jacobian only, no IK
```

A solve takes from seconds (Wrist) to tens of minutes.

2 Words and conventions

These hold for every generated file, in both languages.

- **end-effector-pose** T — a 4×4 homogeneous transform of frame 6 (the last DH frame) in the base frame specifying position and orientation of the end effector.
- **joint-pose** q — a joint vector, one value per joint, in **DH chain order**. The generated code names that order in `JOINT_NAMES`.
- **Units.** Revolute joints are in **radians**. Prismatic joints and all positions are in the **length units of the robot's DH table** (units are implicit in the numbers used in `pvals`).
- **Posture.** (same as joint-pose) Most arms reach one end-effector-pose with several joint-poses — elbow up or down, wrist flipped. IKBT returns **all** of them it can find. Choosing one is your job, because it depends on things IKBT does not know: joint limits, obstacles, where the arm is now.

- **No joint limits** Returned angles are whatever the equations give (for example in $(-\pi, \pi]$). Apply your joint limits
- **Sum-of-angles intermediates** (`th_23` and the like) are computed internally and **not** returned; they are listed in `AUX_NAMES`.

3 Which method did my robot get?

As of Version 3.0, IKBT tries three solution methods in order and stops at the first that works. The first successful solution method decides which function you call.

Symbolic. A complete closed form: an explicit equation for every joint.
Python: `ikin_<Robot>(T)` in `IK_equations<Robot>.py`. C++: `ikin(T)`.

One-variable. No full closed form, but once ONE variable is assumed known, the rest is closed form; a 1-D search finds the right value(s) of that variable. Exact — nothing about the arm is approximated.
Python: `solve_<Robot>(T)` in `IK_onevar<Robot>.py`. C++: `solve(T)`.

Hybrid. When the first two fail, IKBT creates and solves a slightly *simplified* arm in closed form, and uses its solutions as starting points for a 6d numerical solution of your actual arm.
Python: `ikin_<Robot>_approx(T)`, then `refine_seed_<Robot>(T, q_seed)`, in `IK_hybrid_<Robot>.py`. C++: `ikin_approx(T)`, then `refine_seed(T, q_seed)`.

Fallback If the Hybrid method produces an unsatisfactory result, there are many full joint space numerical methods out there (ROS Moveit, robotics-toolbox, etc.). But we do not choose one.

To find out which solution method worked on your robot, any of these works:

- the report's Introduction says "closed form", "ONE-VARIABLE" or "HYBRID";
- the Python file names (above) differ by method;
- the C++ file says it: `grep SOLUTION_PATH CodeGen/Cpp/<Robot>CppCode/<Robot>.cpp`.

Robot names in function names. File names use the robot's own name (`IK_onevarC-Arm.py`); The C++ namespace rules require modification of robot names to eliminate non-identifier characters such as "-". They are turned into `_` (e.g. `C-Arm` → `solve_C_Arm`, `ikbt::C_Arm`).

4 The files, at a glance

All paths are relative to the IKBT root. Everything below except `Cpp_src/` is **generated**: the next solve of that robot overwrites it, so edit a copy, never the original.

file	what	method
LaTex/ik_solution_<R>.tex	the report (Section 7 of this document)	all
CodeGen/Python/IK_equations<R>.py	closed-form IK	symbolic
CodeGen/Python/IK_onevar<R>.py	the 1-D search — **entry point**	one-variable
CodeGen/Python/IK_conditional<R>.py	closed form *given* the assumed variable	one-variable
CodeGen/Python/IK_hybrid_<R>.py	approximate, then correct — **entry point**	hybrid
CodeGen/Python/IK_equations<D>.py	closed form of the simplified arm <D>	hybrid
CodeGen/Python/FK_numeric<D>.py	FK of the simplified arm	hybrid
CodeGen/Python/FK_numeric<R>.py	FK and Jacobian of your robot	all
CodeGen/Cpp/<R>CppCode/<R>.cpp	**all** the C++ for the robot: FK, Jacobian, and the IK	all
Cpp_src/	shared C++ headers and a bundled Eigen (source, not generated)	all
graphs/<R>_graph.txt	the solution graph as data	all
fk_eqns/<R>_pickle.p	FK cache; safe to delete	all

On the hybrid path <D> is the simplified ("derived") arm, named for what was changed — Raven-II's is Raven-II_a_5_0 (DH parameter a_5 set to 0). Its graph file and pickle carry <D> too.

5 Python Code Generation

Each generated module requires only numpy (no IKBT, or sympy imports required) so you can copy CodeGen/Python/ anywhere. When using the one-variable or hybrid methods, the entry point loads its sibling modules (e.g. IK_onevar<R>.py) **by path**, so keep the robot's files together in one directory.

Robot names such as C-Arm are not legal module names, so the examples load modules by path; for a name like Puma, a plain import IK_equationsPuma with CodeGen/Python on sys.path also works.

```
import importlib.util, os
import numpy as np

def load(path):
    name = os.path.basename(path)[:-3].replace('-', '_')
    spec = importlib.util.spec_from_file_location(name, path)
    mod = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(mod)
    return mod
```

Every module also runs as a script (python3 CodeGen/Python/IK_equationsPuma.py) as a self-test.

5.1 Symbolic

```
ik = load('CodeGen/Python/IK_equationsPuma.py')
fk = load('CodeGen/Python/FK_numericPuma.py')
```

```
T = fk.fk_Puma([0.3, -0.5, 0.4, 0.2, 0.7, -0.1])  # a reachable end-effector-pose
sols = ik.ikin_Puma(T)       # list of joint-poses (JOINT_NAMES order), or False
if sols is False:
    print('unreachable')
else:
    for q in sols:
        print(np.abs(fk.fk_Puma(q) - T).max())  # ~1e-16: every one reaches T
```

- Returns **one row per posture that exists at** T, so the count varies with the end-effector-pose; Puma gives up to 8. `False` means none exist.
- `ikin_Puma_labeled(T)` returns the same rows as dicts keyed by joint name.
- Link lengths are module-level constants (`d_1 = 0.6`, ...), visible at the top of the file.

5.2 One-variable

```
ov = load('CodeGen/Python/IK_onevarC-Arm.py')
for s in ov.solve_C_Arm(T):
    s['q']            # the joint-pose, JOINT_NAMES order, ready for FK
    s['error']        # ||dp|| + W_ROT*theta at q; accepted only if <= ACCEPT_TOL
    s['known_value']  # the value of KNOWN_VARIABLE it was found at
    s['branch']       # which branch of the closed form it came from
    s['n_samples']    # the search resolution it settled at
```

- The answers are **exact** — the equations are your real arm's. Typical cost: about 0.1 s per call.
- `[]` means no solution was **found**, which is not quite the same as unreachable.
- If `n_samples == MAX_SAMPLES`, the search was still finding new solutions when it stopped; every returned one is still correct, but some may be missing. Raise `max_samples` for that end-effector-pose: `solve_C_Arm(T, max_samples=16384)`.
- `ikin_<R>_given(T, value)`, in `IK_conditional<R>.py`, is the closed form at one assumed value. It is exact **only** at a correct value, so do not use it directly to command the arm; it is exposed for plotting and diagnosis (`sweep_<R>(T)` gives the raw error curves).
- Running `IK_onevar<R>.py` as a script picks a **random** end-effector-pose and prints the random seed; pass it back (`python3 CodeGen/Python/IK_onevarC-Arm.py 12345`) to repeat that case.

5.3 Hybrid — two calls, on purpose

```
hy = load('CodeGen/Python/IK_hybrid_Raven-II.py')

approx = hy.ikin_Raven_II_approx(T)       # one approximate joint-pose per posture
q, err, iters = hy.refine_seed_Raven_II(T, approx[0])  # correct ONE
```

```
if err <= 1e-9:
    ...                                     # q puts the REAL arm at T

for r in hy.refine_all_Raven_II(T):         # correct every one
    r['index'], r['q'], r['error'], r['converged']
```

- **Solutions of the approximate arms are not answers.** They are good starting points for the numerical method however. Only a corrected joint-pose, solved by the numerical solver reaches T.
- **`refine_seed` takes a joint-pose, not an index**, because choosing where to start *is* choosing the posture — the numerical solver stays near its starting point.
- **Test the error.** `converged` can legitimately be false: the real arm may not reach T in that posture, or at all.
- `refine_all` is for learning, which postures the real arm can actually reach.
- The module's header states how the approximate model was derived (`DH_CHANGES`) and its cost.

5.4 Forward kinematics and Jacobian (every method)

```
fk = load('CodeGen/Python/FK_numericPuma.py')
T  = fk.fk_Puma(q)                  # 4x4 numpy array
J6 = fk.jacobian_Puma(q)            # 6 x 6, EXPRESSED IN FRAME 6
J0 = fk.jacobian_base(T, J6)        # the same, rotated into the base frame
```

Round-tripping each IK answer through `fk_<R>()` is the simplest way to check it. On the symbolic path this module is written only when every parameter has a numeric value.

6 C++ Code Generation

`IKdocs/CPP_API.md` is the full reference — every type and function, and a compiled-and-run program for each method. The short version:

- **One file per robot**, `CodeGen/Cpp/<R>CppCode/<R>.cpp`, everything in `namespace ikbt::<R>`: `fk()`, `jacobian()`, and the IK (`ikin()`, `solve()`, or `ikin_approx()` + `refine_seed()`, as in Section 3). Use it by including it:

  ```
  #include "CodeGen/Cpp/PumaCppCode/Puma.cpp"
  // T is an Eigen::Matrix4d
  std::vector<Eigen::VectorXd> sols = ikbt::Puma::ikin(T);
  ```

- **Build:** `g++ -std=c++17 -O2 myapp.cpp -o myapp -lm`. No `-I`, nothing to install: Eigen is bundled in `Cpp_src/third_party/eigen3/`, and if your build already has Eigen (ROS, CMake) the generated code uses yours.

- **Self-test:**

```
g++ -std=c++17 -O2 -DIKBT_MAIN CodeGen/Cpp/PumaCppCode/Puma.cpp -o ik && ./ik
```

- **Keep the directory where it is relative to `Cpp_src/`.** The robot file reaches the shared headers by a relative `#include` (`../../../Cpp_src/...`). To build elsewhere, copy `Cpp_src/` along with it at the same relative depth.
- Unreachable is an **empty vector**, not `false`. The search parameters, joint-name arrays and the two hybrid calls mirror the Python one for one.

7 The LaTeX report

`LaTex/ik_solution_<R>.tex` is a self-contained document. To make the PDF:

```
cd LaTex && pdflatex ik_solution_Puma.tex
```

It contains the DH parameters, forward kinematics, the unknowns in solution order, every solution equation (long expressions are shortened by naming pieces K_1, K_2, ... defined just above them), the solution graph (which variable depends on which), the table of complete solutions, the Jacobian, and — on the one-variable and hybrid paths — sections saying what was assumed or simplified and how the numerical step works. The last section lists the files produced.

The equations are the same ones the generated code evaluates, so the report is the way to read or cite the solution by hand. `pdflatex` leaves `.aux`, `.log` and `.out` files beside the PDF; they can be deleted.

8 Validating The Results with Closed Loop Testing

The full validation involves:

1. Choose one or more test joint poses, θ_t, at which the algorithm will be validated.
2. Assuming the Forward Kinematics is correct, use the FK to compute the reachable end-effector pose corresponding to the test joint pose(s)

$$T_t = FK(\theta_t)$$

3. For each T_t: Evaluate the inverse kinematics solution(s) to get

$$\theta_{si} = IK(T_t)$$

   (where i indexes among multiple solutions)

   (a) Compute a test end effector pose, T_{ei} for each θ_{si}

   (b) Compute a distance measure between the input test pose, T_t and end-effector pose (T_{ei}).

(c) Confirm that all solutions produced have error below a small numerical threshold

(d) Confirm that one of the solutions is equal to θ_t.

To perform this closed-loop validation, run these commands from the IKBT root.

`python3 -m scripts.numerical_closed_loop_sol_check <R> --keep`

Perform the above steps for Python code

`python3 -m scripts.cpp_closed_loop_check <R> --keep`

Perform the above steps for generated C++ code.

`python3 -m scripts.check_solution_sets <R>`

Perform the above steps for the internal `sympy` solution code.

The flag `--keep` checks the files already written instead of solving the robot again. `<R>` indicates the name of your robot.

More on testing IKBT

`IKdocs/TESTING.md` describes these and the rest of the test suite in detail.

9 Other outputs

Some additional outputs can be generated but are not automatic with `ikSolver` or with closed-loop testing. None of them solves the robot again. Note use of . and lack of the `.py`; needed for invocation as modules.

how	what
`python3 -m scripts.tube_map <R>`	Produce the solution graph drawn as a "London Tube Map" — one coloured line per complete solution, one station per version of each variable (`graphs/<R>_tube.svg`). Reads only `graphs/<R>_graph.txt`. On the hybrid path use the simplified arm's name.
`python3 fkOnly.py <R>`	Forward kinematics only: `LaTex/fk_equations_<R>.tex`, `CodeGen/Python/FK_equations<R>.py` (a readable dump of the symbolic FK), and `CodeGen/Cpp/<R>CppCode/FK_numeric<R>.h`. Use it for a robot you need only FK for; it does not touch IK files.
`python3 -m scripts.bt_diagram`	Draws the behavior tree itself, as `.svg` files (`IKdocs/IKBT_BT_p1..p3.svg`). (an open source tool to view and edit `.svg` is Inkscape.)

10 Regenerating and cleaning up

- **Re-run the solve** to regenerate everything for a robot; it overwrites that robot's files.
- `CodeGen/cleanCodeGenOutput` (run from the root) deletes all generated Python and C++.
- `LaTex_src/cleanLaTexFolder` (run from the root) deletes all generated reports.
- `fk_eqns/` is a cache of forward kinematics; delete any or all of it at any time. The next solve recomputes it, which costs only time. Changing a robot's DH table refreshes its cache automatically.
- `Cpp_src/` and `LaTex_src/` are source, not output: the cleanup scripts never touch them, and neither should you.

11 When something looks wrong

you see	it means
`ikin_<R>()` returns `False` (Python) or an empty vector (C++)	no posture exists at that end-effector-pose. Check it is in the workspace, and in the DH table's units.
fewer joint-poses than the report's solution table has rows	normal: rows whose `acos`/`asin`/`sqrt` argument leaves its domain at this end-effector-pose are postures that do not exist there, and are dropped.
`XXXXX` in the C++, and `g++` stops on it	a parameter has no `pvals` entry. Deliberate: a missing link length is never silently defaulted. Add it to `ik_robots.py` and re-solve, or type the value in.
no `FK_numeric<R>.py` after a symbolic solve	same cause: FK needs every parameter to have a value. The IK is still fine.
hybrid: `converged` false / large error	the real arm does not reach T from that starting joint-pose. Try another, or the arm's current joint-pose; if none converges, T may be unreachable.
one-variable: `n_samples == MAX_SAMPLES`	the search had not settled; raise `max_samples`.
one-variable: `[]` for an end-effector-pose you believe reachable	raise `n_samples`/`max_samples`, and plot `sweep_<R>(T)` to see the error curves.
a solve takes very long	normal for hard arms (minutes). `python3 ikSolver.py <R> --perf` reports sympy timings and names each slow call as it happens.